Scilab 數值模擬範例
Elo rating system

2011/10/13

Chih-Han Lin 林致翰

clin@ltl.iams.sinica.edu.tw
or r99245002@ntu.edu.tw

What is Elo rating system?

一種用來計算多人參與時個別選手對弈水平 (relative skill) 的評價系統，比賽者透過一對一的競賽，根據雙方的原始分數來決定預設勝率，根據比賽結果與預設勝率來決定賽後雙方分數的增減。

現代許多線上遊戲的 Rank 與隨機配隊系統也採用基於 Elo rating system 改良的演算法來衡量玩家的實力，或者根據 Elo score 來替 Solo 玩家組隊（3v3, 5v5），將大多數的玩家勝率控制在 50% 來避免玩家遇到實力不對稱的對手（被慘電或者電爆對手）而失去遊玩樂趣

目前較具知名度採用 Elo 評價來 quene 的遊戲：
LOL (League of Legends), StarCraft2

Elo 評分系統

獲勝的可能性與
等級分呈指數分佈

$$E_{\text{win}}(R_A) = \frac{Q_A}{Q_A + Q_B}, \qquad E_{\text{win}}(R_B) = \frac{Q_B}{Q_A + Q_B}$$

$$E_{\text{win}}(R_A) = \frac{1}{1 + 10^{-(R_A - R_B)/400}}, \qquad Q_A = 10^{(R_A/400)}, \quad Q_B = 10^{(R_B/400)},$$

由勝負決定分數升降

$$R'_A = R_A + K \times (E_p - E(R_A)), \quad E_p = 0 \ \ (\text{loss}), \qquad E_p = 1 \ \ (\text{win})$$

ex. $R_A = 900, \quad R_B = 800, \quad K = 32,$

$$E_{\text{win}}(R_A) \simeq 0.64, \qquad E_{\text{win}}(R_B) = 1 - E_{\text{win}}(R_A) = 0.36,$$

$$R'_A = 900 + 32 \times (1 - 0.64) = 911.5 \qquad R'_B = 800 + 32 \times (0 - 0.36) = 788.5$$

遇上等級分低 100 分的對手，預測勝率約 64%，獲勝可加 11 等級分（視 K 值而定）
遇上等級分低 200 分的對手，預測勝率約 76%

模擬流程

亂數製造 n 位遊戲者
設定真實實力（指數分佈）`Pos(2,:)`
給定 Elo 分（起始值為真實實力平均值）`Pos(1,:)`
紀錄勝場數 `Pos(1,:)`

for loop

`mysort(Pos)`
`Solofight(Pos,k)`

儲存 `Pos`

分析數據

`PosFinal=mysort(Pos)`
根據遊戲者 Elo 分排序，使陣列 index 等於排名（Elo 分越高者放在陣列越前面處）

`PosFinal=Solofight(Pos,K)`
挑取陣列中相鄰者競賽，依照真實實力決定勝負；依照Elo 分紀錄調整賽後分數，紀錄勝場

排序

將 Elo 加上亂數讓同分者排序略有變動

```
function PosFinal=mysort(PosInitial)
    lengthPos=size(PosInitial,2);
    PosInitial(1,:)=PosInitial(1,:)+rand(1,lengthPos);

for i=1:lengthPos,
    maxPos=find(PosInitial(1,:)==max(PosInitial(1,:)));
    PosFinal(:,i)=PosInitial(:,maxPos(1));
    PosInitial(1,maxPos)=0;
end
PosFinal(1,:)=floor(PosFinal(1,:));

endfunction
```

1 搜索陣列找第一個極大值的位置

2 將第一個極大值填入新陣列

最後讓 Elo 分無條件進位

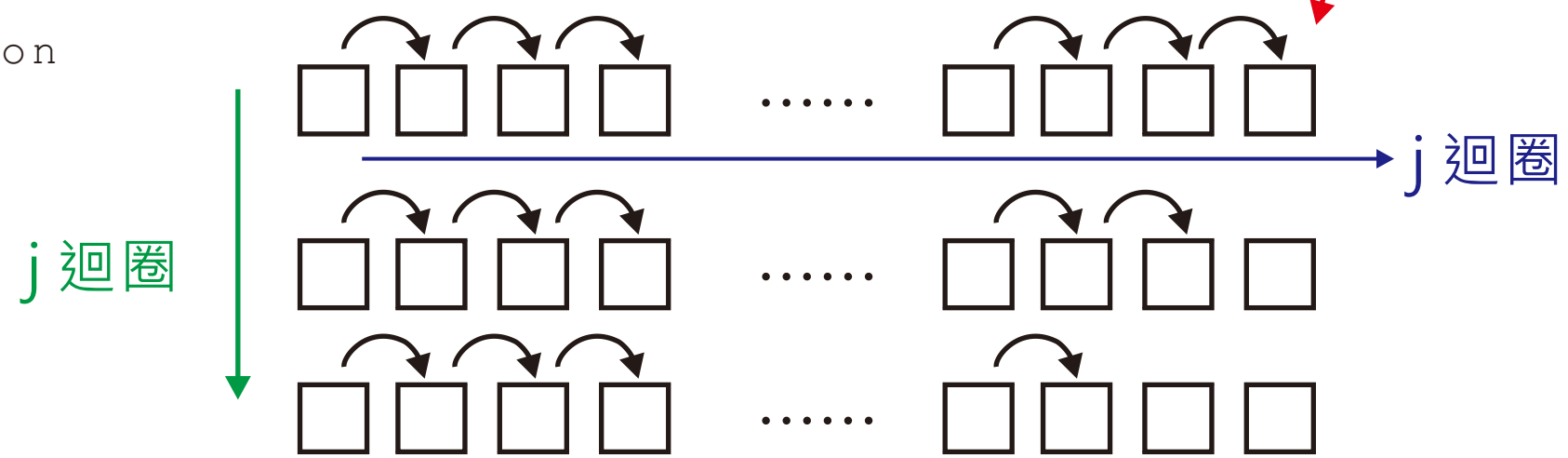
3 將原陣列該極大值處歸零
下一個迴圈判定極值時才不會被此值干擾

排序函數效率測試

```
function PosFinal=mysortbubble(PosInitial)
    lengthPos=size(PosInitial,2);
    PosFinal=PosInitial;
    //PosInitial(1,:)=PosInitial(1,:);+rand(1,lengthPos)-0.5;

for i=1:(lengthPos-2),
    for j=1:(lengthPos-i),
        if PosFinal(1,j)<PosFinal(1,j+1)  then
           temp=PosFinal(:,j);
           PosFinal(:,j)=PosFinal(:,j+1);
           PosFinal(:,j+1)=temp;
        end
    end
end
endfunction
```

使用氣泡排序法

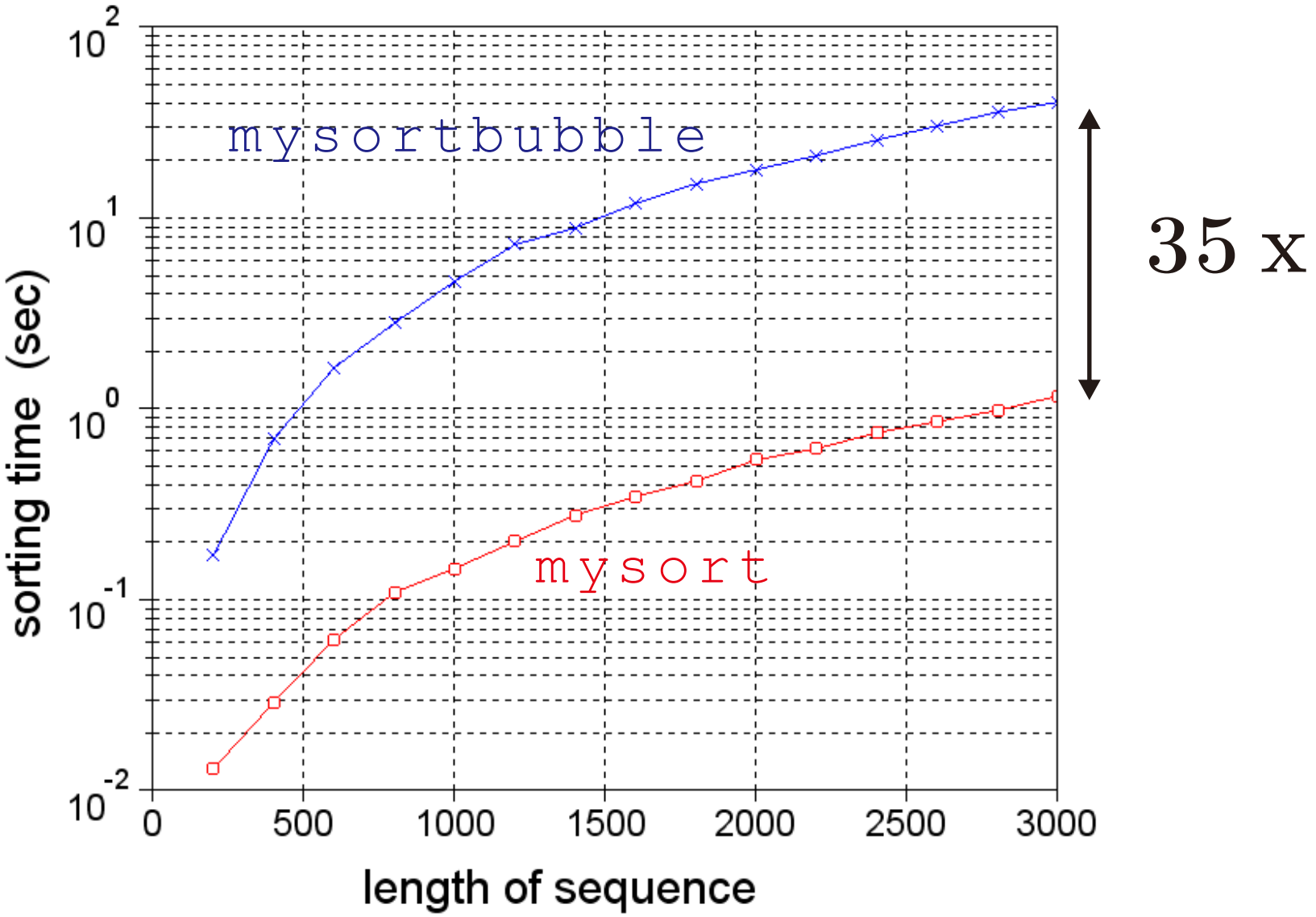


設定被整理序列之長度，分別估計 mysort 與 mysortbubble 的耗時

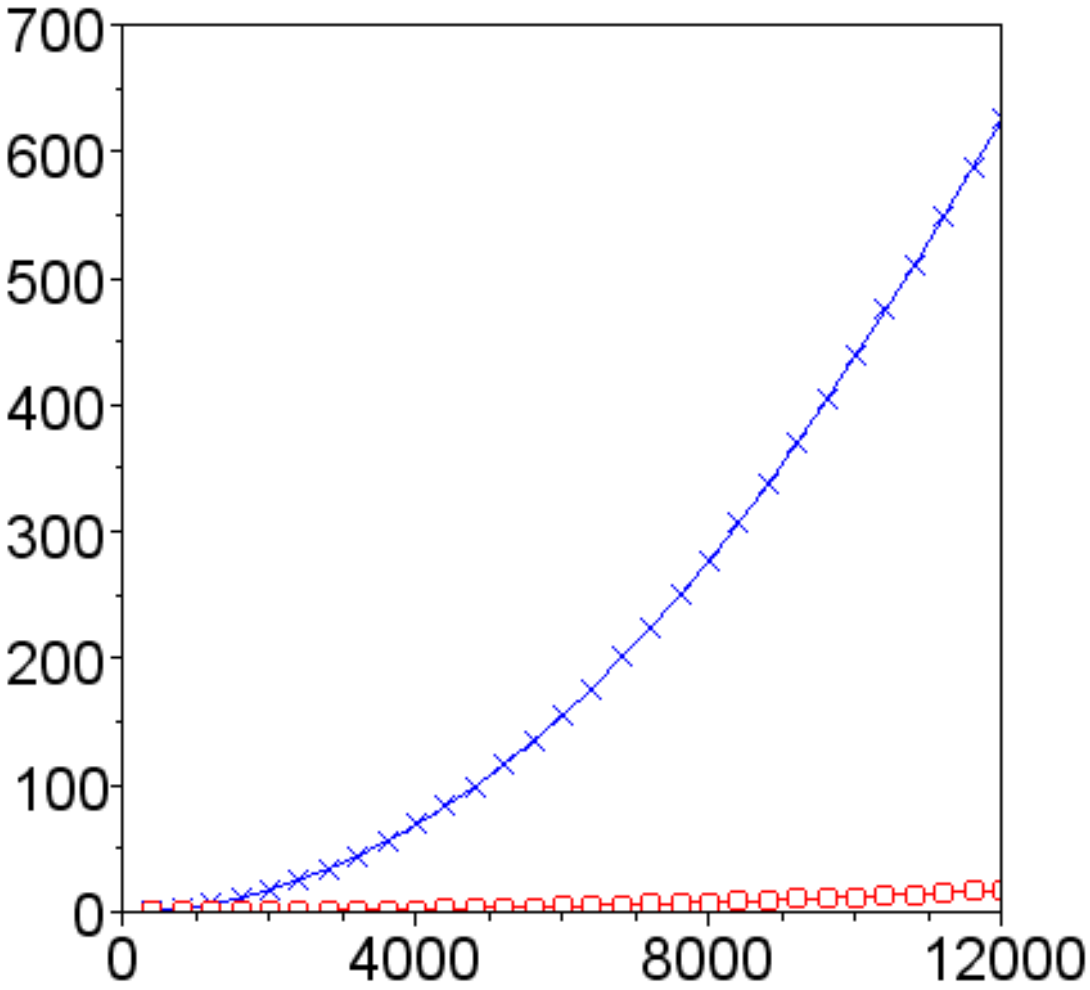
```
clear a; clear qtime; clear qtimeBubble;
step=20;
stepSize=50;
a=ceil(100*rand(3,step*stepSize+1));
for i=1:step
    tic()
    aSortBubble=mysortbubble(a(:,1:(stepSize*i)));
    qtimeBubble(1,i)=toc();
    printf('cycle %d complete, time= %f sec \n',i,qtimeBubble(1,i));
end
for i=1:step
    tic()
    aSort=mysort(a(:,1:(stepSize*i)));
    qtime(1,i)=toc();
    printf('cycle %d complete, time= %f sec \n',i,qtimeBubble(1,i));
end

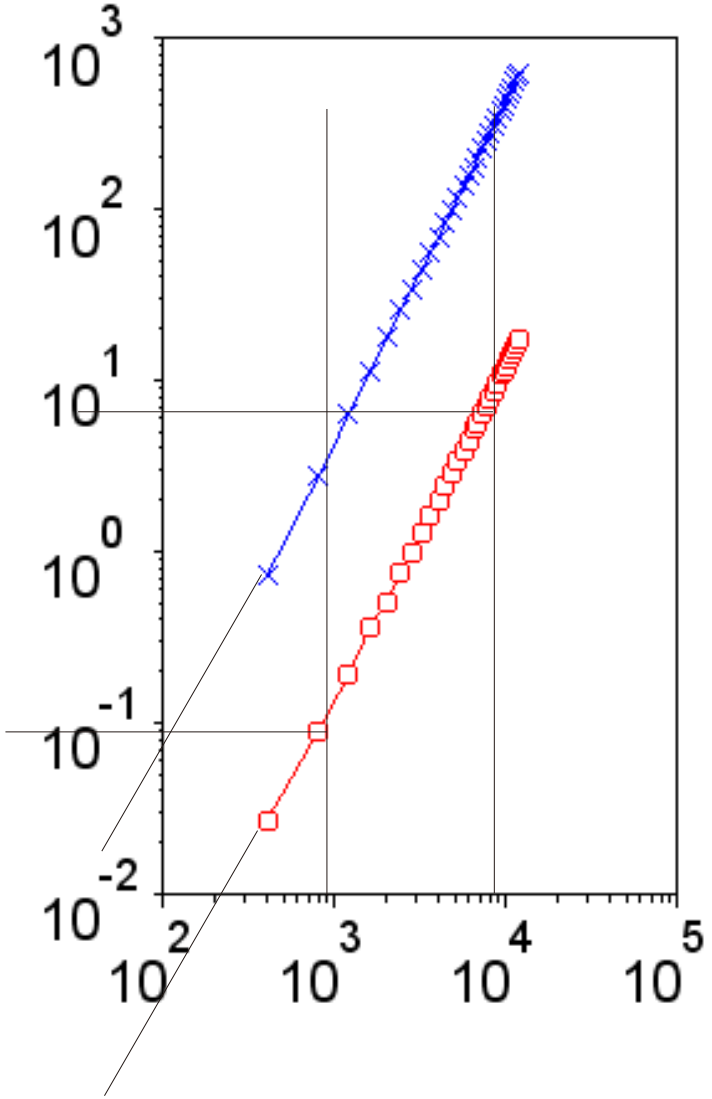
plot((1:step)*stepSize,qtimeBubble(1,:),'bx-')
plot((1:step)*stepSize,qtime(1,:),'ro-')
```

mysort 的執行速度仍與 bubble sorting 同一個 order
(雖然比較快，還有大幅改進的空間)

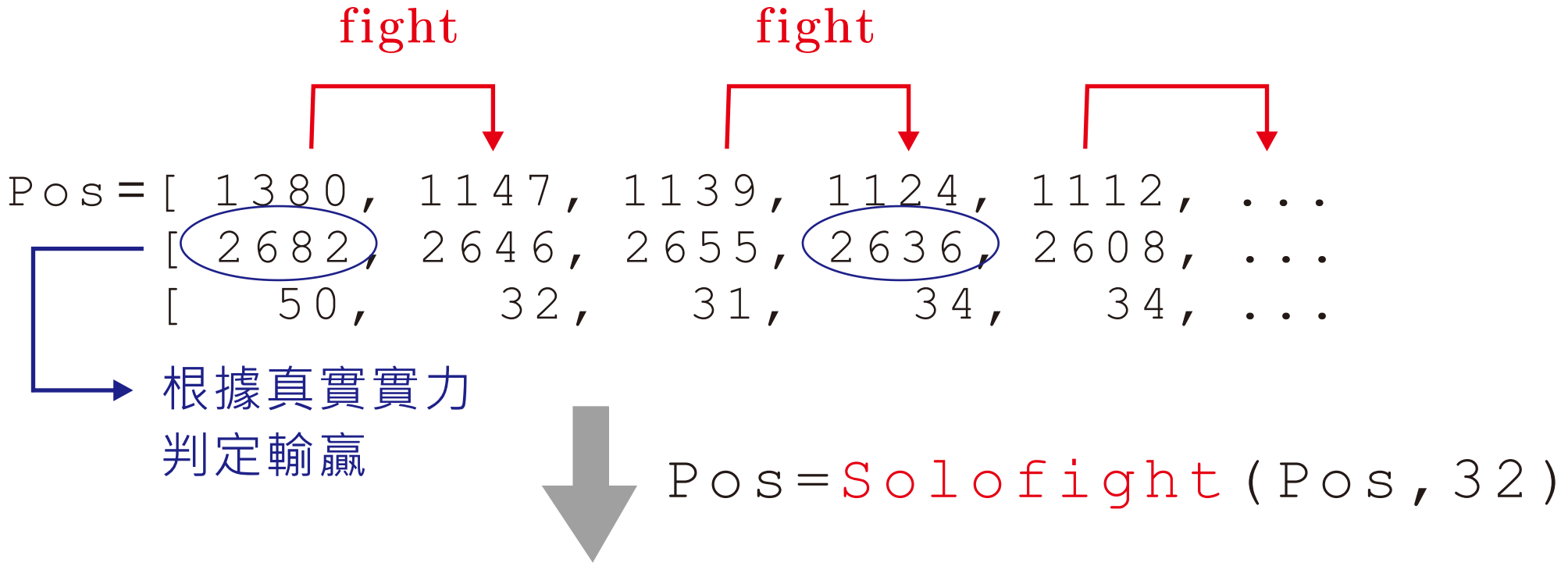


t~(size_a)^2


700
600
500
400
300
200
100
0
0
4000
8000
12000


10^3
10^2
10^1
10^0
10^-1
10^-2
10^2
10^3
10^4
10^5

競賽


fight
fight
Pos=[1380, 1147, 1139, 1124, 1112, ...
[2682, 2646, 2655, 2636, 2608, ...
[50, 32, 31, 34, 34, ...
根據真實實力
判定輸贏
Pos=Solofight(Pos,32)
昇 降 昇 降 降
Pos=[1386, 1140, 1154, 1108, 1095, ...
[2682, 2646, 2655, 2636, 2608, ...
[51, 32, 32, 34, 34, ...
Solofight不執行排序

```
function PosFinal=Solofight(PosInitial,k)
    lengthPos=size(PosInitial,2);

        for i=1:2:lengthPos-1,
        RankdiffReal=PosInitial(2,i)-PosInitial(2,i+1);
        RankdiffFake=PosInitial(1,i)-PosInitial(1,i+1);

        Ewin=1/(1+10^(-RankdiffFake/400));
        Eloss=1-Ewin;
```

RankdiffFale 紀錄 Elo 分差 (調分與勝率計算)

RankdiffReal 紀錄 真實實力差 (判定輸贏)

判定勝負

調 Elo 分

勝場

```
        if RankdiffReal>0 then
            PosFinal(1,i)=PosInitial(1,i)+k*(1-Ewin);
            PosFinal(1,i+1)=PosInitial(1,i+1)-k*Eloss;
            PosFinal(3,i)=PosInitial(3,i)+1;
            PosFinal(3,i+1)=PosInitial(3,i+1);
        else
            PosFinal(1,i)=PosInitial(1,i)-k*Ewin;
            PosFinal(1,i+1)=PosInitial(1,i+1)+k*(1-Eloss);
            PosFinal(3,i)=PosInitial(3,i);
            PosFinal(3,i+1)=PosInitial(3,i+1)+1;
        end
    end
    PosFinal(2,:)=PosInitial(2,:);
endfunction
```

真實實力不做變更

簡單測試

```
k=35;
fighter=32000;
Pos(2,:)=ceil(2700*%e.^(-3*rand(1,fighter)));
Pos(1,:)=ceil(sum(Pos(2,:))/fighter)*ones(1,fighter);
Pos(3,:)=zeros(1,fighter); //wins
Pos=mysort(Pos);
```

製造 32k 名參賽者，其真實實力呈指數分佈（expert mode, 儘管母體實力可能是常態分佈，但假設只有實力超過兩個標準差的人才敢參賽）




```
histplot(100,Pos(2,:))
```

```
for i=1:50,
    timer;
    Pos=Solofight(Pos,32);
    Pos=mysort(Pos);
    q(1,i)=timer();
    printf('finish  %d  cycles,  cpu time: %f  sec \n'...
    ,i,q(1,i))
    clf();
    plot(1:fighter,Pos(3,:)/i);
    xs2png(gcf(),'fig'+string(i)+'.png');

end

printf('total consumed time: %f  secs',sum(q));
```

排序+競賽

即時監控

勝率圖

```
 PosReal(1,:)=Pos(2,:);
 PosReal(2,:)=Pos(1,:);
 PosReal=mysort(PosReal);
 temparray(1,:)=PosReal(1,:);
 PosReal(1,:)=PosReal(2,:);
 PosReal(2,:)=temparray(1,:);
 PosDiff=Pos()-PosReal;
```

利用真實實力排序

PosReal(2,:)

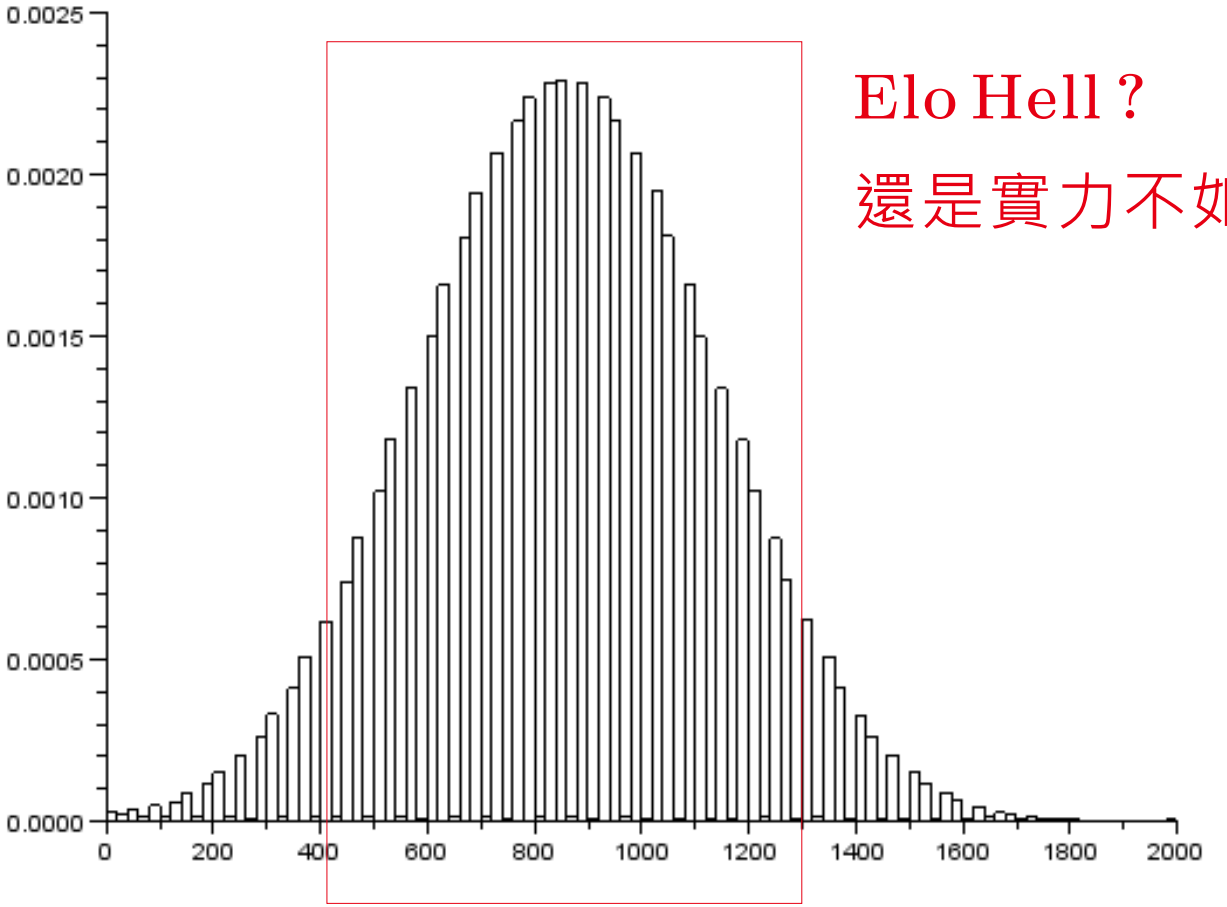
為該位置真實實力

PosDiff(2,:)

可衡量系統判定能力

少量場次後整體 Elo 分的分佈大致呈高斯分佈（300 Games）

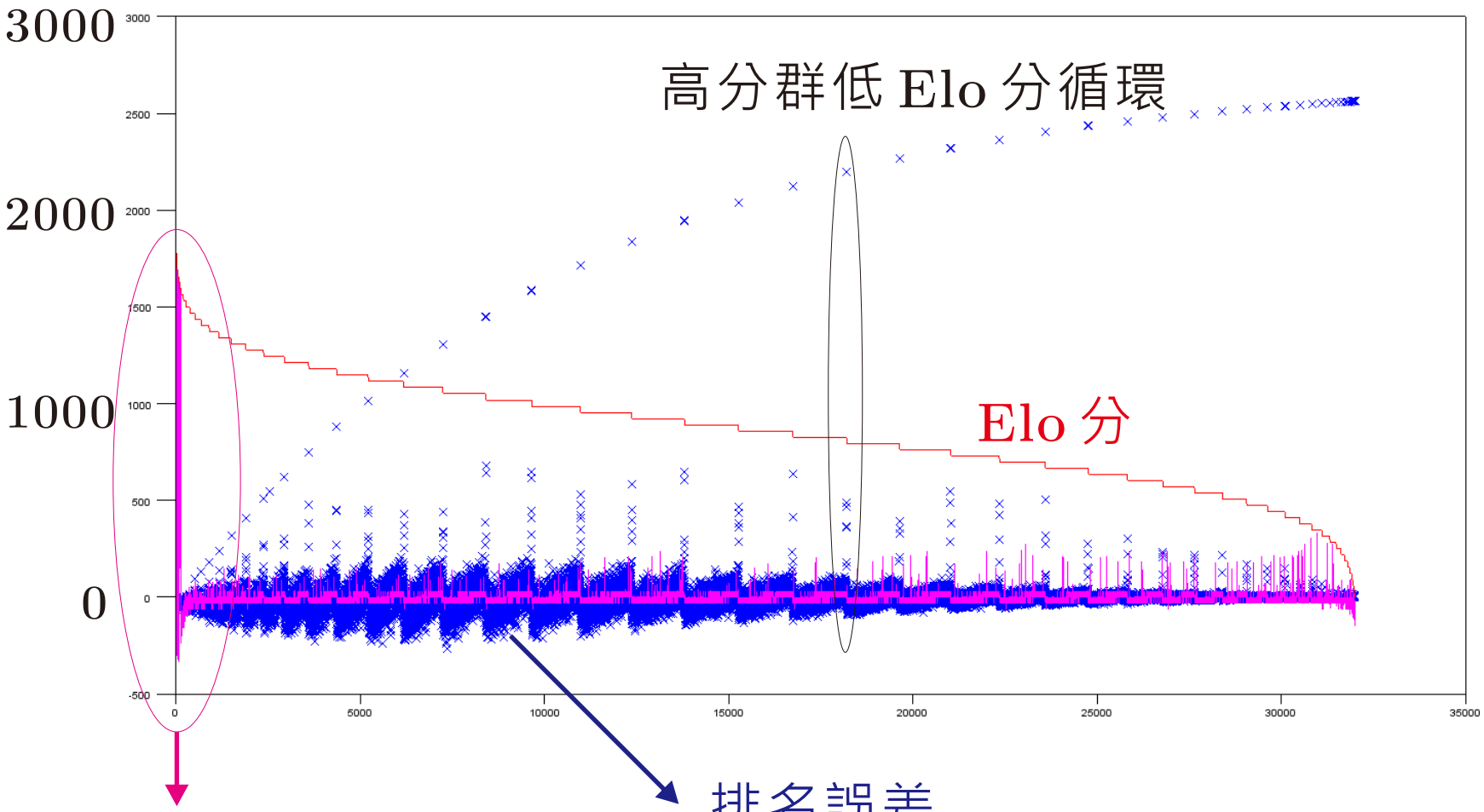
```
histplot(100,Pos(1,:))
```



Elo Hell？

還是實力不如人？

```
plot(PosDiff(1,:),'m-')
plot(Pos(1,:),'r-')
plot(PosDiff(2,:),'bx')
```



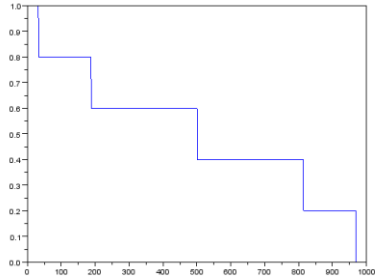
真實實力差
（賽制導致高分群低估）

排名誤差
（絕大多數少於 100）

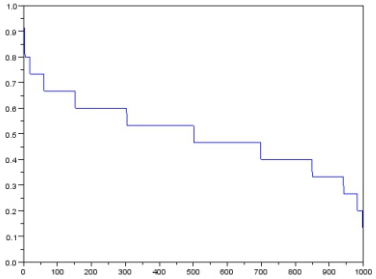
勝率控制

取 1000 players，觀察勝率變化（50 Games）

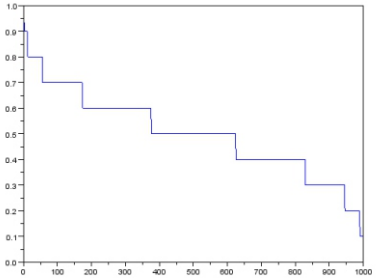
5 games



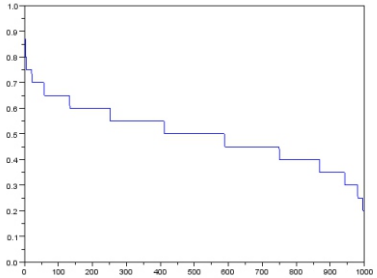
10 games



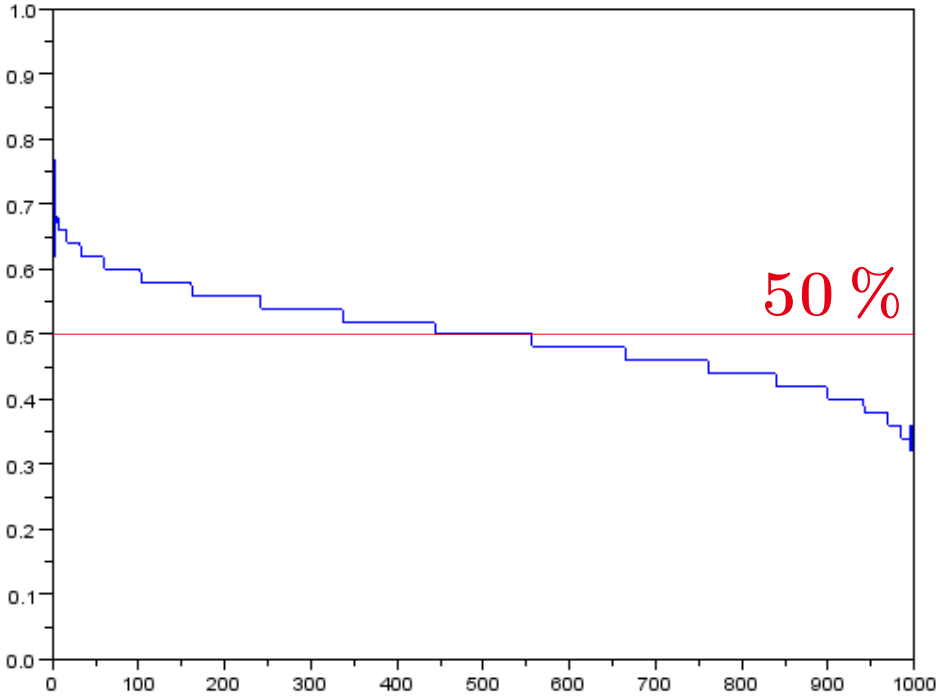
15 games



250 games

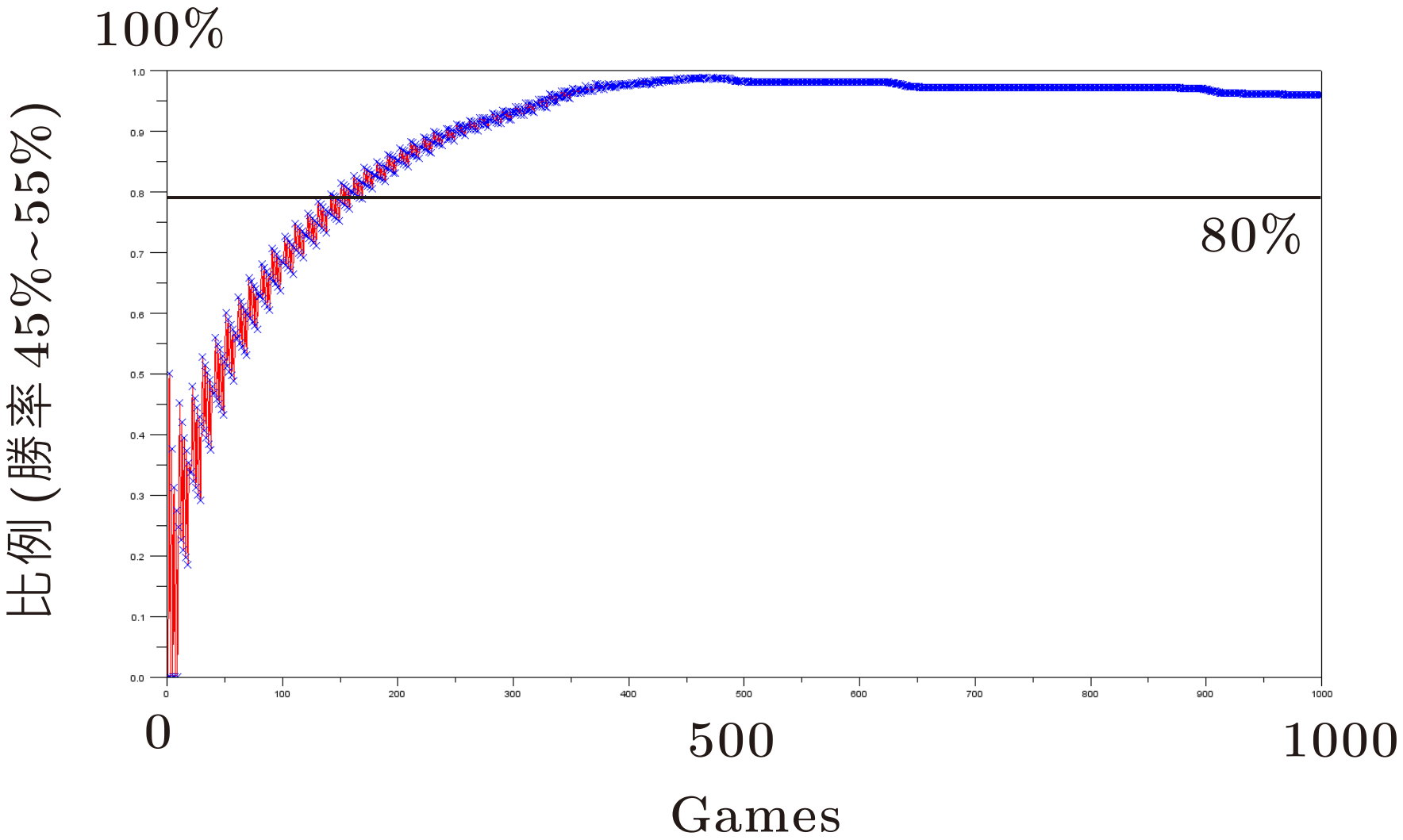


50 games

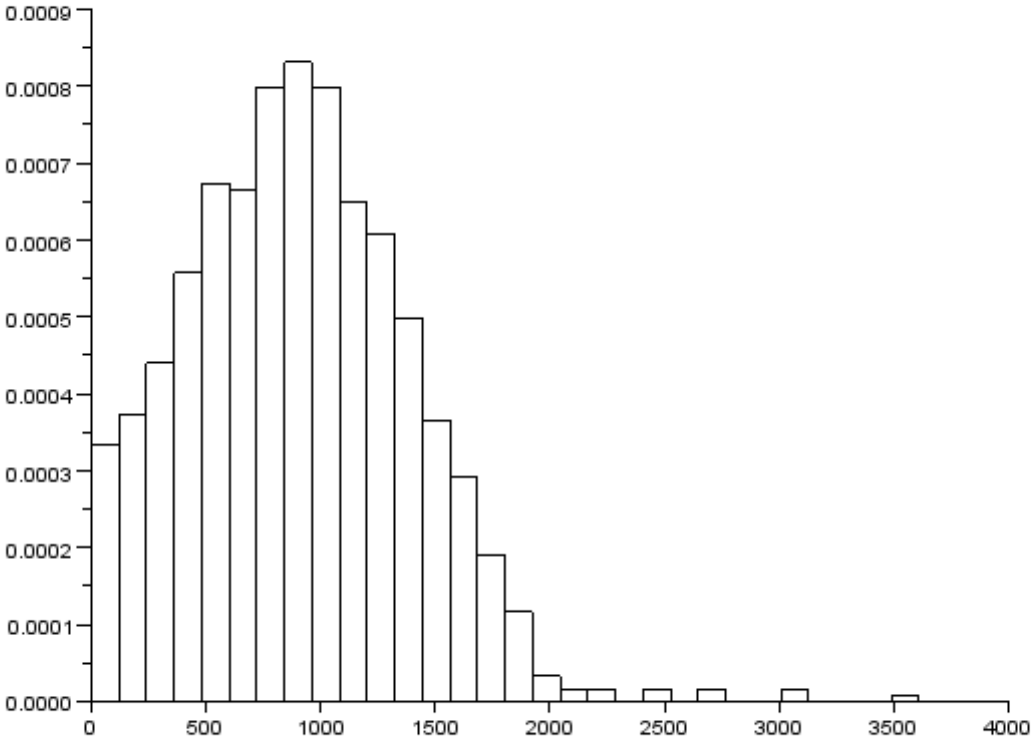


After 50 Games , 52% 的參賽者勝率介於 45% 至 55%

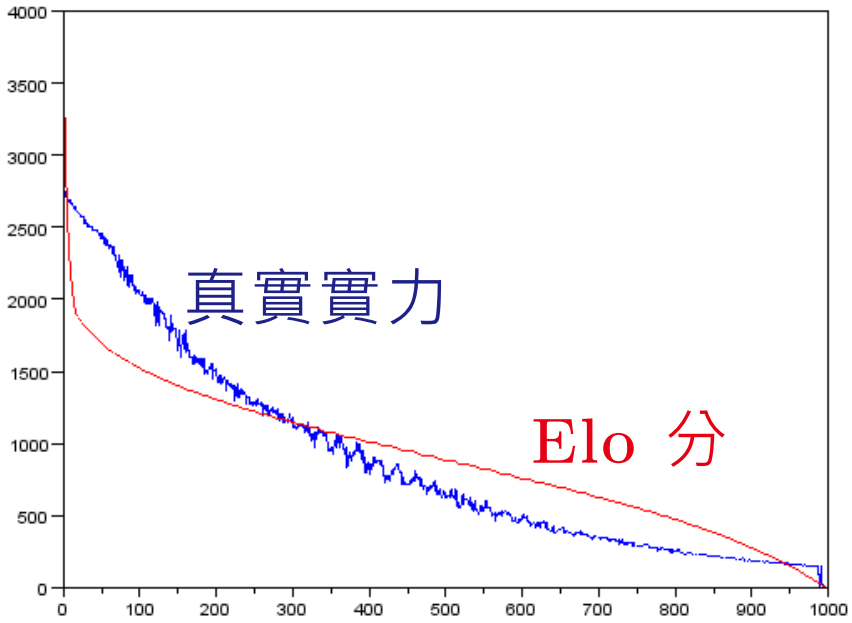
取 1000 players，觀察勝率介於 45% ~55% 人數比例之變化
(1000 Games)



1000 players，每人經歷 1000 Games (avg: 687)

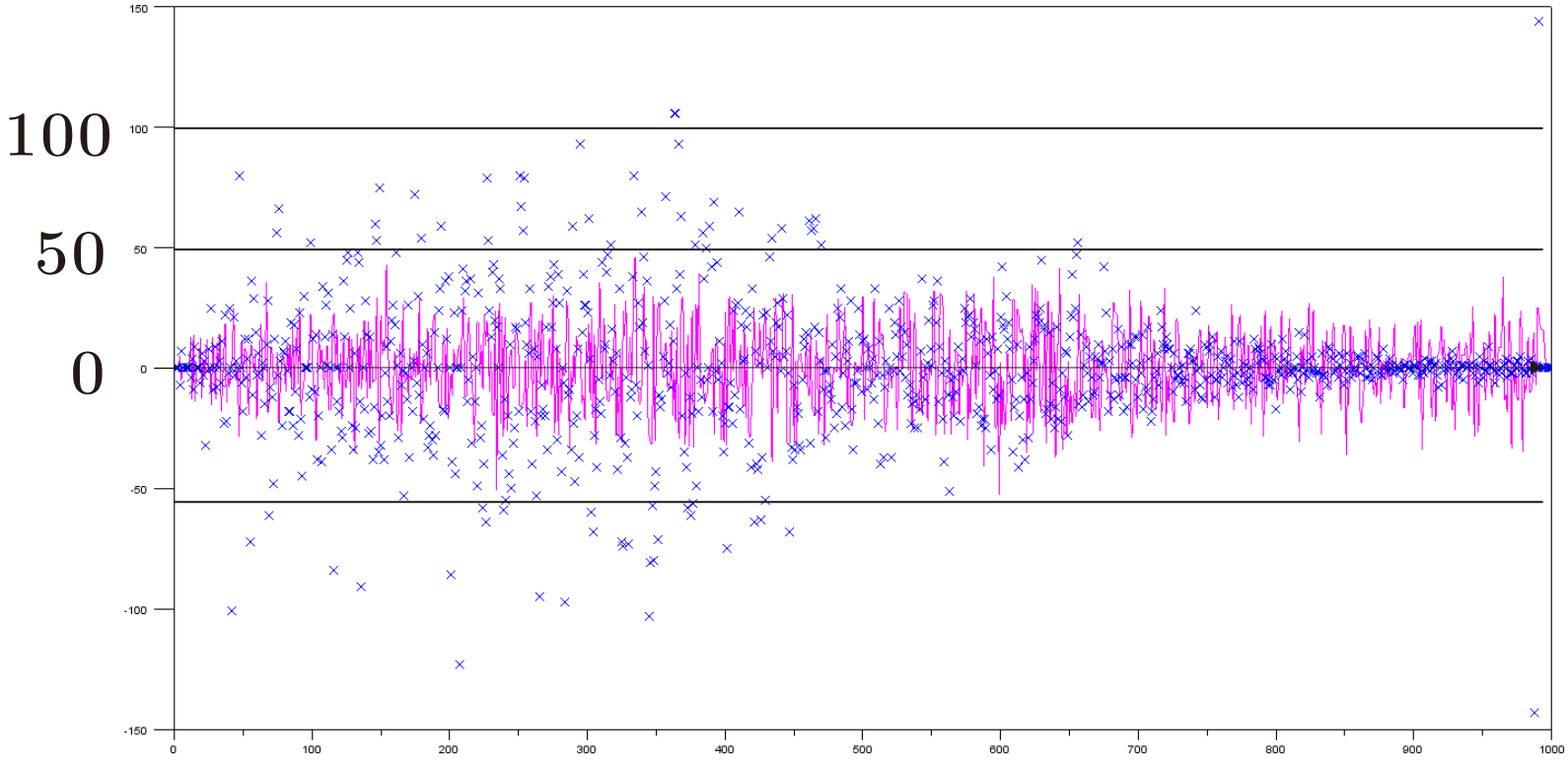


Elo 分之分佈

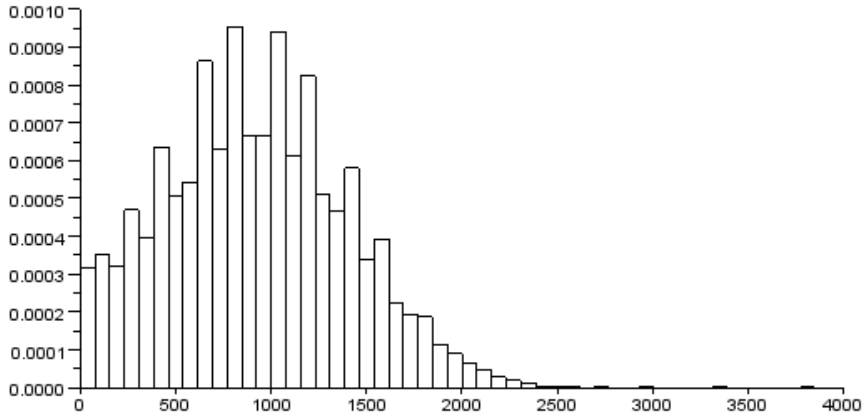


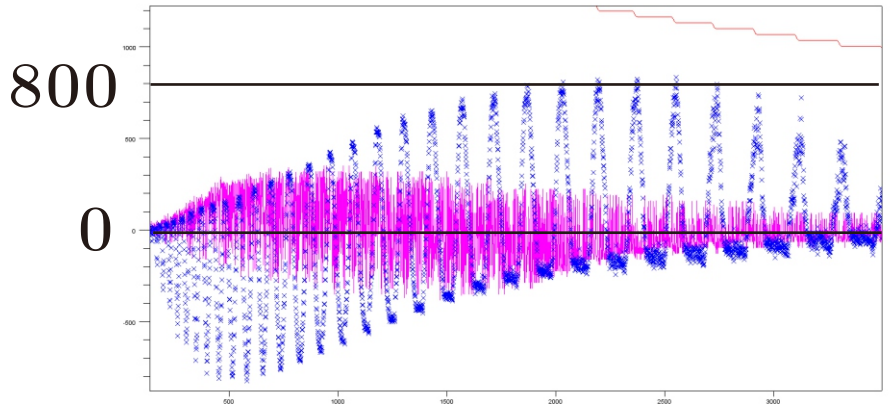
若場次足夠多，Elo 分的分佈仍會接近真實實力分佈（指數分佈）

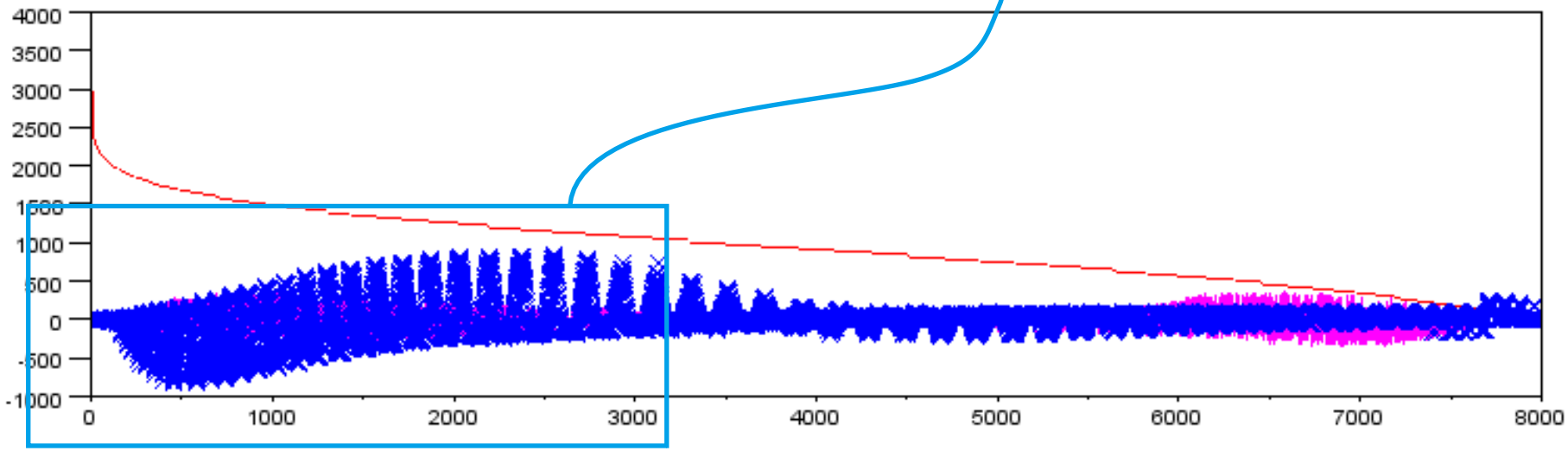
高 Elo 值排名誤差較大
(絕大多數少於 50)



8K players，每人經歷 1000 Games (avg: 923)







快速排序

不論是 bubble sort 或者透過 max 指令來架構的 select sort，計算時間複雜度約略與序列長度 n 成平方正比 O(n^2)；事實上 Scilab 有內建的快速排序法 gsort 可以利用

```
--> a=ceil(6*rand(1,6))

   a = 3.    6.    6.    3.    4.    1.

-->[asort, order]=gsort(a)

order  =   2.    3.    5.    1.    4.    6.
asort  =   6.    6.    4.    3.    3.    1.

--> b=a(order)

b = 6.    6.    4.    3.    3.    1.
```

order 紀錄了排序前的位置資訊

```
function PosFinal=mysortquick(PosInitial)

[PosFinal(1,:), order]=gsort(PosInitial(1,:));
```

以 elo 分進行排序

```
counter=1;
for i=order,
    PosFinal(2:3,counter)=PosInitial(2:3,i);
    counter=counter+1;
end
endfunction
```

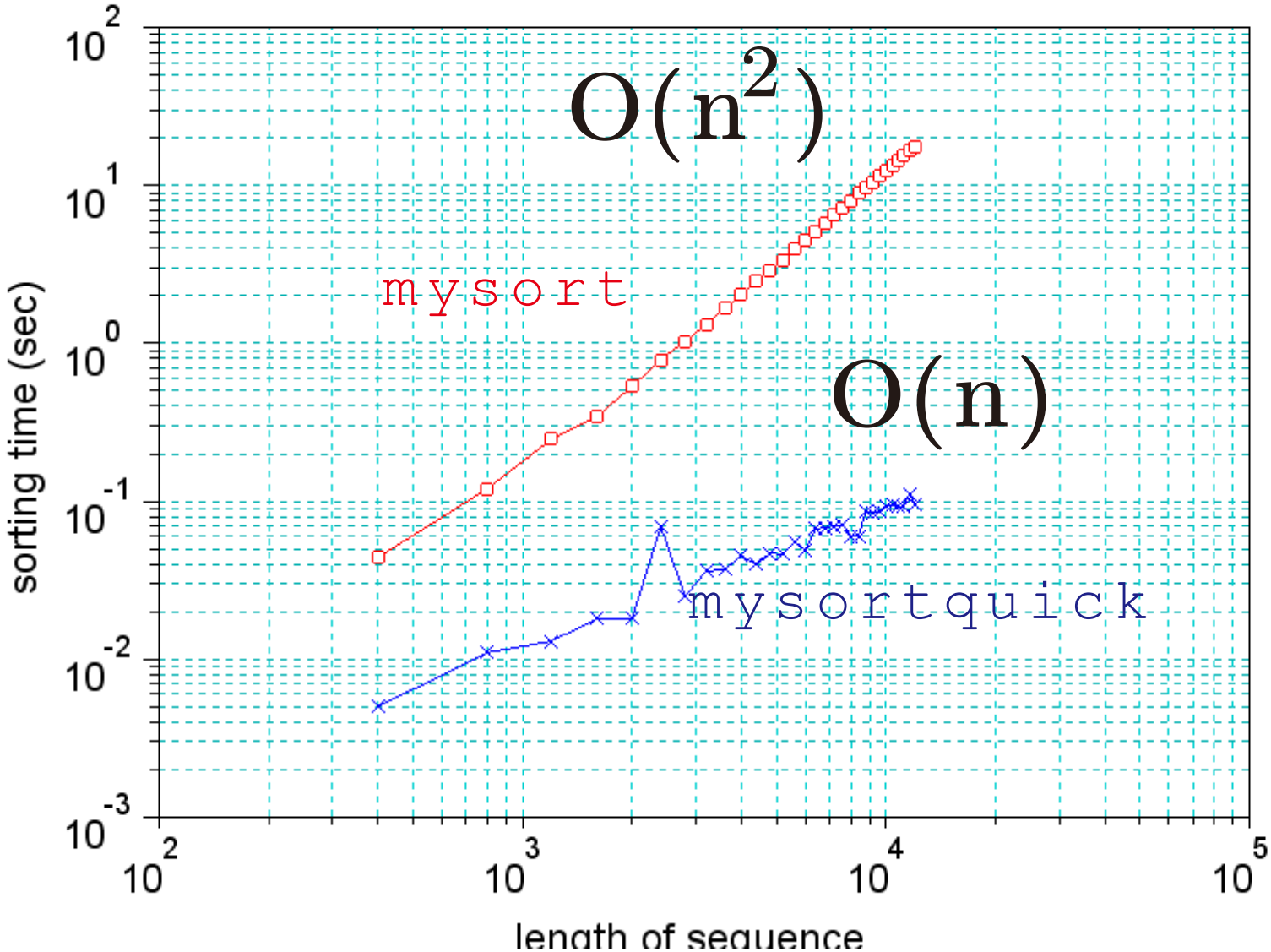
根據 order 來將對應的勝場數、真實實力填入新陣列

等價寫法

```
for i=1:size(PosInitial,2),
    PosFinal(2:3,i)=PosInitial(2:3,order(1,i));
end
```

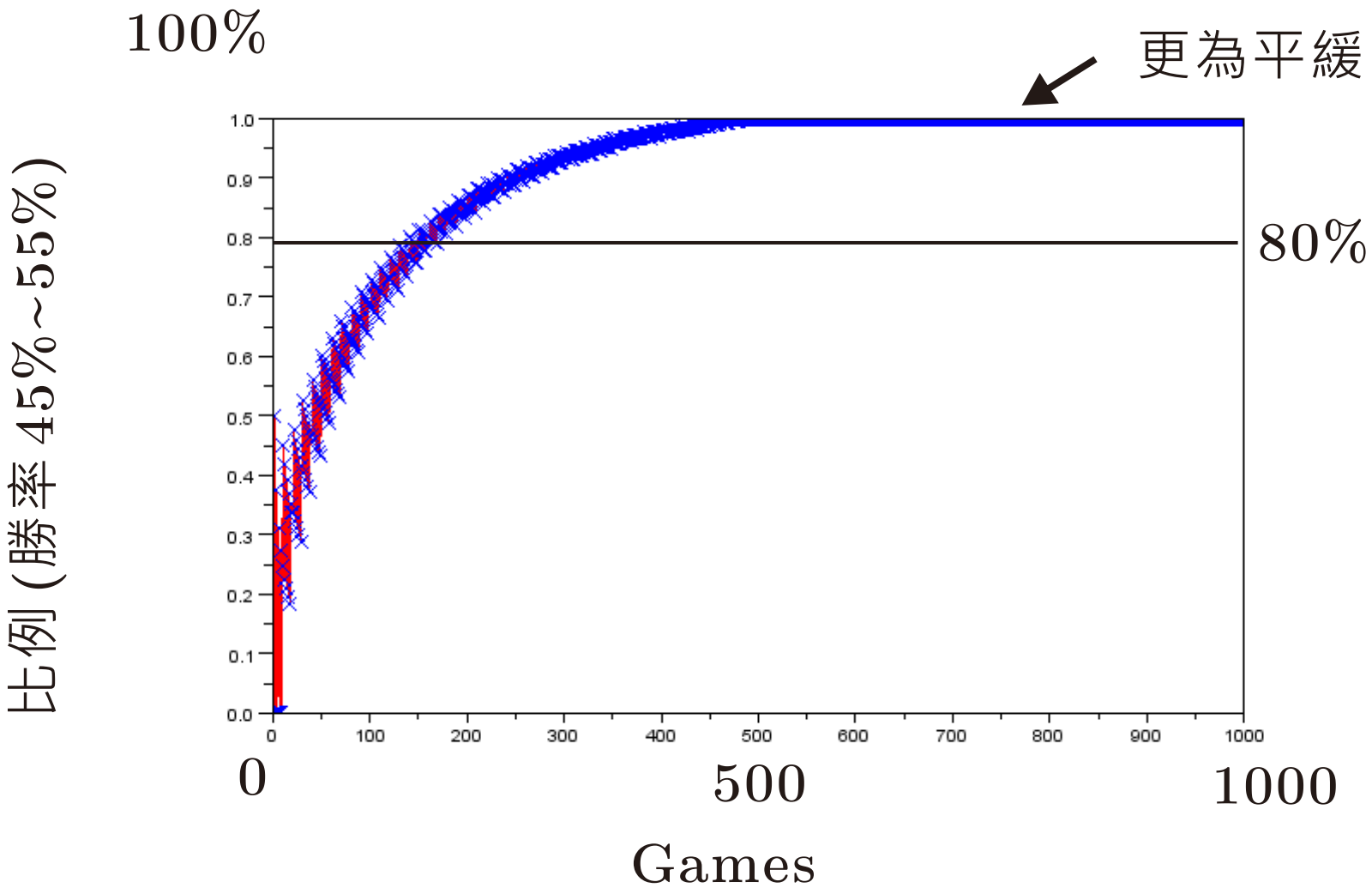
快速排序

8 k player –> quick sort 只需要 mysort 的 1% 計算時間



取 1000 players，觀察勝率介於 45% ~55% 人數比例之變化

使用 mysortquick （1000 Games）



修正重點

避免強者在低分區互鬥 ex. 排定對戰加入隨機性
校正 Elo 與真實實力 ex. 起始真實實力最大值定為 Elo 極值
校正 K 值 ex. 高 Elo 分群採用低 K 值穩定排名

進階測試：

根據設定經過大量對戰製造穩態排名資料庫

1) 新增一名遊戲者，給予起始 Elo 分，與資料庫對戰，模擬可能的勝場數
2) 假設該遊戲者實力隨場次變化，觀察勝場數變化
3) 討論組隊對戰 (3v3, 5v5) 的配對方式